

Taller Integrador de Bajo Nivel en C++ y Arquitectura Dual-Remote

Manuel Ascencio Padilla (Grupo 99)

Escuela de Educación Secundaria N° 99 "Juana Azurduy" - Vicente López

Laboratorio de Programación — 5° Año

Prof. Mansilla Muñoz York

25 de septiembre de 2026

Introducción

El desarrollo de software en entornos técnicos exige una comprensión profunda de la gestión de memoria y la estructuración modular del código, así como el dominio de herramientas distribuidas para el control de versiones. El propósito de este informe es analizar la implementación práctica de tres Retos de programación en lenguaje C++, complementados con un flujo de trabajo de sincronización simultánea hacia repositorios remotos dobles (*Dual-Remote* en GitHub y GitLab).

A través del análisis del código fuente y los trazados en memoria RAM, se demuestra la manipulación directa de la Pila (*Stack*), la eficiencia temporal en algoritmos de búsqueda lineal $O(n)$ y la alteración física de celdas mediante el desreferenciamiento de punteros. Asimismo, se documenta la resolución de conflictos técnicos de transferencia bajo protocolos HTTPS y tokens de acceso personal.

Desarrollo

Reto 1: Suma Recursiva y Control de Stack Frames

La recursividad permite resolver problemas complejos dividiéndolos en subproblemas idénticos. En el módulo sumaRecursiva(int n), la función se invoca de manera autorreferencial reduciendo el parámetro en una unidad por iteración. Para evitar un desbordamiento de pila (*Stack Overflow*), se estableció una condición de parada

o Caso Base:

$$n \leq 0 \implies \text{retorna } 0$$

Cada llamada reserva un *Stack Frame* en la RAM. Cuando $n = 5$, la pila apila cinco marcos sucesivos hasta alcanzar el Caso Base, momento en que la pila se desapila sumando secuencialmente los retornos intermedios hasta devolver el valor 15.

Reto 2: Búsqueda Secuencial en Arreglos y Direccionamiento Contiguo

Un arreglo es una estructura de datos contigua en la memoria física. La búsqueda secuencial implementada recorre un vector de 10 elementos para localizar un valor

clave (18). Al identificar la coincidencia en el índice [6], el programa ejecuta la instrucción break, finalizando el ciclo de manera optimizada con una complejidad temporal en el mejor de los casos de $O(1)$ y en el peor de $O(n)$.

Al imprimir la dirección física mediante el operador de dirección `&vectorDatos[i]`, se verifica la contigüidad espacial. Cada celda de tipo int ocupa exactamente 4 bytes en la RAM, difiriendo en +4 hexadecimal respecto de su predecesora.

Reto 3: Intercambio Físico de Memoria mediante Punteros

El paso por valor genera copias locales en el ámbito de la función que no afectan a las variables invocadoras. Para efectuar una modificación real en las celdas del main(), la función intercambiarValores(int* ptrA, int* ptrB) recibe las direcciones de memoria hexadecimales mediante el operador &.

Dentro de la función, la variable `temporal` actúa como búfer intermedio.

Utilizando el operador de desreferenciación *, se lee el contenido apuntado y se reescribe directamente sobre las celdas asignadas a las variables X e Y, invirtiendo sus valores (100 y 500) de forma definitiva en el espacio de direcciones físicas.

Conclusión

La ejecución del taller demostró la importancia crítica del control de memoria en C++. La recursividad requiere condiciones de corte estrictas para salvaguardar la estabilidad del sistema operativo. Por su parte, la combinación del operador de dirección (&) y desreferenciación (*) constituye la herramienta fundamental para el desarrollo de programas eficientes y de bajo nivel.

Finalmente, la integración de una arquitectura de control de versiones Dual-Remote garantiza la redundancia y el respaldo del código fuente en múltiples proveedores de la nube, asegurando estándares profesionales de entrega e integración continua.

Referencias

Kernighan, B. W., & Ritchie, D. M. (1988). *El lenguaje de programación C* (2.^a ed.). Prentice Hall.

Mansilla Muñoz, Y. E., P. (2026). *Guía docente de replicación y protocolo anti-excusas — Actividad 6*. Escuela de Educación Secundaria Técnica N° 99 "Juana Azurduy".

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.